WEEK 4 LAB ACTIVITY

METADATA | WALKTHROUGH

1.0 EXIFTOOL

Command: exiftool ocean.jpg

We use **exiftool** to extract metadata from the image file.

Metadata is hidden information stored inside the image such as file details, creation data and additional embedded information that cannot be seen normally.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ exiftool ocean.jpg
ExifTool Version Number         : 13.25
File Name                       : ocean.jpg
Directory                       : .
File Size                       : 43 kB
File Modification Date/Time     : 2026:04:08 11:05:04-04:00
File Access Date/Time           : 2026:04:08 14:21:41-04:00
File Inode Change Date/Time     : 2026:04:08 14:21:36-04:00
File Permissions                : -rw-rw-r--
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : inches
X Resolution                    : 72
Y Resolution                    : 72
Profile CMM Type                : Little CMS
Profile Version                 : 2.1.0
Profile Class                   : Display Device Profile
Color Space Data                : RGB
Profile Connection Space        : XYZ
Profile Date Time               : 2012:01:25 03:41:57
Profile File Signature          : acsp
Primary Platform                : Apple Computer Inc.
CMM Flags                       : Not Embedded, Independent
Device Manufacturer             :
Device Model                    :
Device Attributes               : Reflective, Glossy, Positive, Color
Rendering Intent                : Perceptual
Connection Space Illuminant     : 0.9642 1 0.82491
Profile Creator                 : Little CMS
Profile ID                      : 0
Profile Description             : c2
Profile Copyright               : IX
Media White Point               : 0.9642 1 0.82491
Media Black Point               : 0.01205 0.0125 0.01031
Red Matrix Column               : 0.43607 0.22249 0.01392
Green Matrix Column             : 0.38515 0.71687 0.09708
Blue Matrix Column              : 0.14307 0.06061 0.7141
Red Tone Reproduction Curve     : (Binary data 64 bytes, use -b option to extract)
Green Tone Reproduction Curve   : (Binary data 64 bytes, use -b option to extract)
Blue Tone Reproduction Curve    : (Binary data 64 bytes, use -b option to extract)
Comment                         : THIS IS THE HIDDEN FLAG
Image Width                     : 640
Image Height                    : 425
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 640×425
Megapixels                      : 0.272
```

From the output, a hidden comment was also found:

THIS IS THE HIDDEN FLAG

This shows that images can contain hidden data within metadata fields such as comments.

2.0 HEXEDITOR

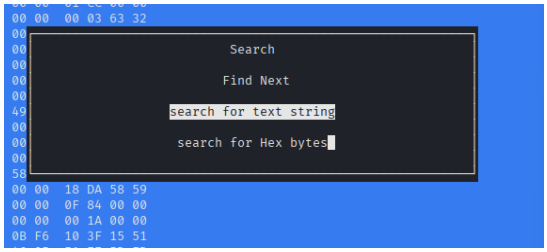
Command: hexeditor computer.jpg

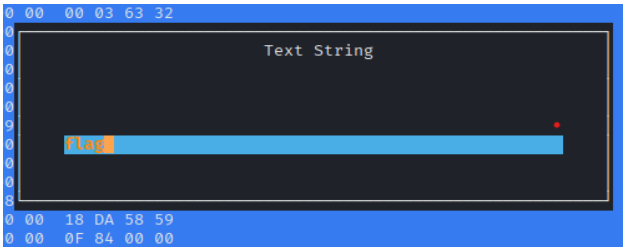
We use **hexeditor** to view the raw binary content of the file.

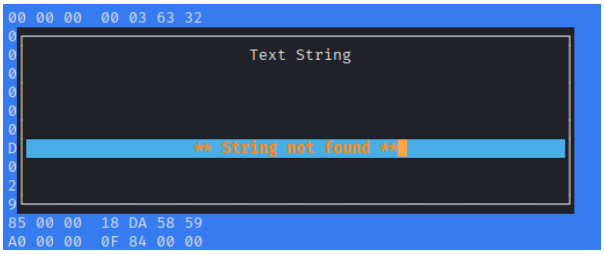
It allows us to analyze both hexadecimal values and ASCII-readable text inside the image, which may reveal hidden or embedded information.

From the ouput, we can see that the hexeditor displays two sections:

- Left side shows hexadecimal values
- Right side shows ASCII-readable text







No hidden flag was identified in the file after performing keyword searches and analyzing the ASCII-readable output.

This indicates that the file mainly contains standard metadata and no embedded hidden message.

3.0 BINWALK

Command 1: binwalk dog.jpg

We use **binwalk** to scan the image file for any hidden or embedded files. This tool helps detect file signatures that may be stored inside another file.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ binwalk dog.jpg

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             JPEG image data, JFIF standard 1.01
88221         0x1589D         Zip archive data, at least v1.0 to extract, compressed size: 21, uncompressed size: 21, name:
 hidden_text.txt
88384         0x15940         End of Zip archive, footer length: 22
```

The output shows that dog.jpg contains normal JPEG image data, and it also reveals a ZIP archive embedded inside the image at offset 88221.

It also indicates the presence of a file named hidden_text.txt.

Command 2: binwalk -e dog.jpg

We use **-e** to automatically extract the hidden files detected by binwalk.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ sudo apt install unzip
[sudo] password for najihah:
unzip is already the newest version (6.0-29).
unzip set to manually installed.
The following packages were automatically installed and are no longer required:
  amass-common        libmongocrypt0  libtheoraenc1          python3-gpg                          python3-protobuf
  libbluray2          libnet1         libudfread0            python3-kismetcapturebtgeiger        python3-zombie-imp
  libbson-1.0-0t64    libplacebo349   libx264-164            python3-kismetcapturefreaklabszigbee  samba-ad-dc
  libjs-jquery-ui     libportmidi0    libxml2                python3-kismetcapturertl433          samba-ad-provision
  libjs-underscore    librav1e0.7     python3-bluepy         python3-kismetcapturertladsb         samba-dsdb-modules
  libmongoc-1.0-0t64  libtheoradec1   python3-click-plugins  python3-kismetcapturertlamr
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 619

┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ binwalk -e dog.jpg

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
88221         0x1589D         Zip archive data, at least v1.0 to extract, compressed size: 21, uncompressed size: 21, name: hidden_text.tx
t

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented
```

Unfortunately, the command produced a warning and failed to extract the hidden file automatically.

This means binwalk successfully detected the hidden ZIP archive, but it could not extract it directly, so manual extraction was needed.

Command 3: dd if=dog.jpg of=hidden.zip bs=1 skip=88221

We use the **dd** (data duplicator) command to manually extract the hidden ZIP file starting from the offset value found by binwalk.

- if=dog.jpg → input file
- of=hidden.zip → output file
- bs=1 → copy byte by byte
- skip=88221 → skip the first 88221 bytes and start extracting from there

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ dd if=dog.jpg of=hidden.zip bs=1 skip=88221
185+0 records in
185+0 records out
185 bytes copied, 0.0021448 s, 86.3 kB/s
```

A new file named hidden.zip was successfully created.

Command 4: unzip hidden.zip

We use **unzip** to extract the contents of the hidden ZIP file.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ unzip hidden.zip
Archive:  hidden.zip
 extracting: hidden_text.txt
```

The file hidden_text.txt was extracted successfully.

This confirms that the hidden ZIP archive contains another file inside it.

Command 5: cat hidden_text.txt

We use **cat** to display the content of the extracted text file.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ cat hidden_text.txt
THIS IS A HIDDEN FLAG
```

The output displays "**THIS IS A HIDDEN FLAG** "

4.0 STRINGS

Command: strings computer.jpg

We use the **strings** command to extract human-readable text from a binary file. This helps identify any embedded text or information without analyzing raw hexadecimal data.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ strings computer.jpg
JFIF
ICC_PROFILE
lcms
mntrRGB XYZ
9acspAPPL
-lcms
desc
^cprt
wtpt
bkpt
rXYZ
gXYZ
bXYZ
rTRC
@gTRC
@bTRC
@desc
text
XYZ
-XYZ
XYZ
XYZ
XYZ
curv
#*%%*525EE\
#*%%*525EE\
L89|
%M80sy<?=
}/On
pq9<
L@ h
x?#.
2-0@
_Wgc
Ncli
u←?
^Rz4
b`  `
tt;]
n`)9
HBcrr
#Fl\
Jv^8L
vY;l
)NNU
cLdaV|<
!]Ug
```

The command displays multiple readable strings such as:

- JFIF
- ICC_PROFILE
- RGB XYZ
- lcms

Some random or unreadable characters are also shown.

No hidden flag or meaningful secret message was found.

5.0 FILE (CHECK FILE)

Command 1: file solitaire.exe

We use the **file** command to determine the actual type of a file based on its content, not just its file extension.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ file solitaire.exe
solitaire.exe: PNG image data, 640 x 449, 8-bit/color RGBA, non-interlaced
```

The output shows that the file is PNG image data, even though it has a .exe extension.

This shows that the file extension does not match the actual file type. Although it is named as an executable file, it is actually an image. This can be used to trick users into thinking the file is something else.

Command 2: file rubiks.jpg

We use the **file** command to determine the actual type of a file based on its content, not just its file extension.

```
┌──(najihah㉿kali)-[~/Downloads/image/image]
└─$ file rubiks.jpg
rubiks.jpg: PNG image data, 609 x 640, 8-bit/color RGBA, non-interlaced
```

The output shows that the file is PNG image data, even though it has a .jpg extension.

This shows that the file extension does not match the actual file type. Although it is named as a JPG image, it is actually a PNG image. This can confuse users because the file name does not reflect its real format.